UNIVERSIDAD TÉCNICA DE MANABÍ
Fundada en 1952

Desarrollo De Sistemas Informáticos

Actividad # 1. Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

Nombre: Jorge Luis Camacho Pinargote
Paralelo: A

Tutor: Ing. Jorge Arun Zambrano Cedeño, Mg.

Apertura: martes, 22 de septiembre de 2026, 06:25

Cierre: sábado, 17 de octubre de 2026, 23:24

- **Enlace al github:**

https://github.com/jcamacho4852/Actividad1-Rescate-y-Adopci-n-Animal

Índice

1. Introducción y Descripción del Escenario 3
2. Análisis de Requerimientos 3
2.1. Identificación de Actores 3
2.2. Requerimientos Funcionales 3
2.3. Requerimientos No Funcionales 3
3. Modelado UML 4
3.1. Diagrama de Casos de Uso 4
3.2. Diagrama de Clases 5
4. Implementación de Patrones de Diseño 6
4.1. Patrón Creacional: *Factory Method* 6
4.2. Patrón Estructural: *Singleton* 6
4.3. Patrón de Comportamiento: *Observer* 6
5. Conclusiones 6

1. Introducción y Descripción del Escenario

El presente informe detalla el diseño de la arquitectura lógica para un **Sistema de Control de Rescate y Adopción Animal**. Este software busca automatizar y optimizar los procesos de registro de animales rescatados, el control de su historial médico, la gestión de adopciones y el seguimiento por parte de los administradores y rescatistas de una fundación o refugio.

El propósito de este desarrollo bajo el paradigma orientado a objetos es garantizar una solución escalable, mantenible y robusta, aplicando buenas prácticas de ingeniería de software, estándares de modelado UML y un flujo de trabajo profesional mediante control de versiones.

2. Análisis de Requerimientos

2.1. Identificación de Actores

- **Ciudadano / Adoptante:** Usuario interesado en consultar el catálogo de animales disponibles e iniciar una solicitud de adopción.
- **Rescatista:** Personal encargado de registrar los ingresos de nuevos animales rescatados al sistema y actualizar su estado de salud inicial.
- **Administrador:** Usuario con privilegios totales para gestionar el catálogo general, aprobar o rechazar solicitudes de adopción y generar reportes operativos.

2.2. Requerimientos Funcionales

- **RF01:** El sistema debe permitir registrar el ingreso de un animal rescatado indicando sus características físicas, especie, raza y ubicación de rescate.
- **RF02:** El sistema debe permitir llevar un control del historial médico y tratamientos de cada animal.
- **RF03:** El ciudadano debe poder registrarse en la plataforma y enviar una solicitud de adopción adjuntando los requisitos solicitados.
- **RF04:** El administrador debe poder evaluar, aprobar o rechazar las solicitudes de adopción basándose en los criterios de la institución.

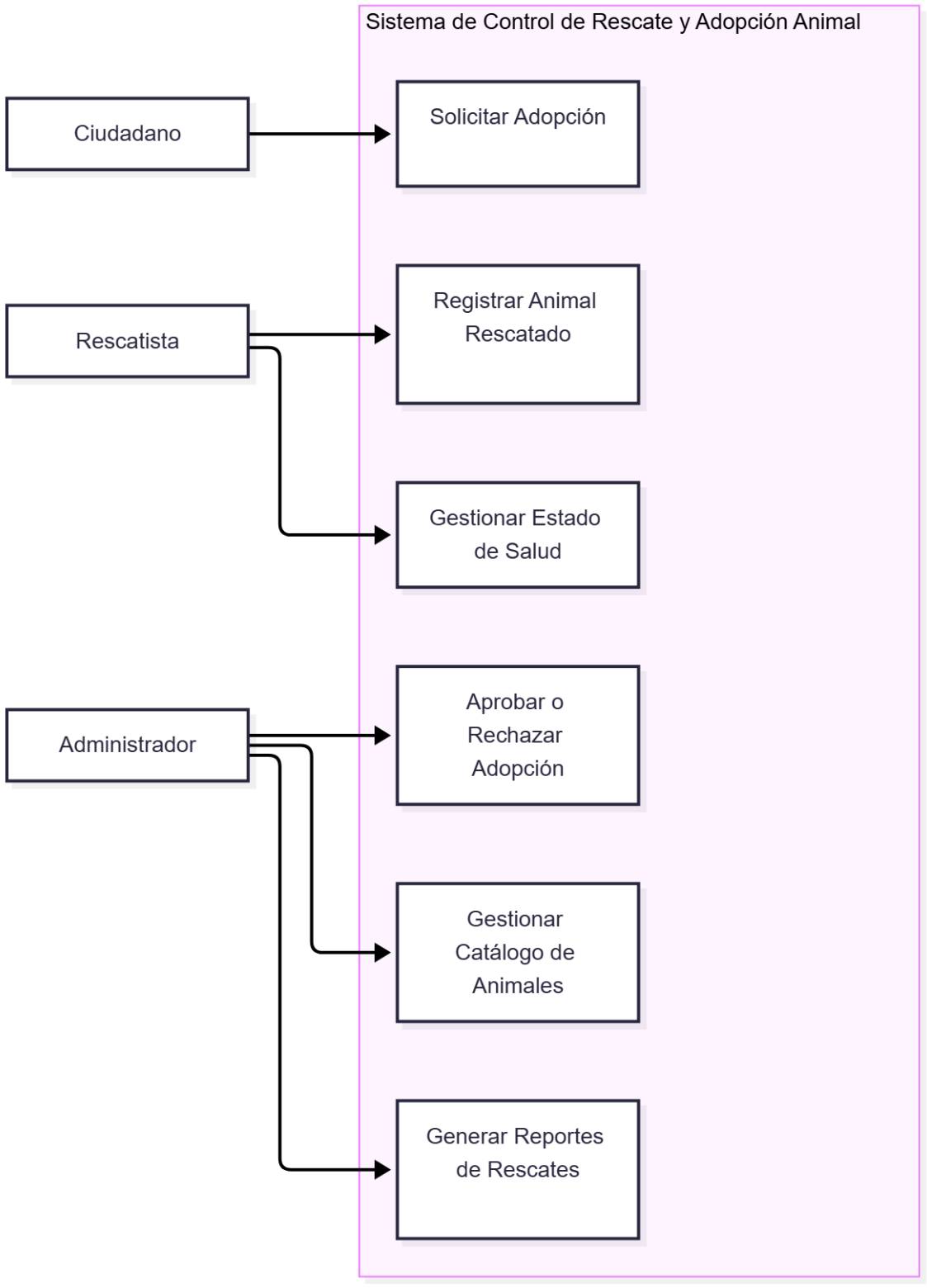
2.3. Requerimientos No Funcionales

- **RNF01:** La base de datos debe garantizar la integridad y confidencialidad de la información personal de los adoptantes.
- **RNF02:** El sistema debe ser altamente disponible y accesible mediante una interfaz web intuitiva o aplicación móvil.

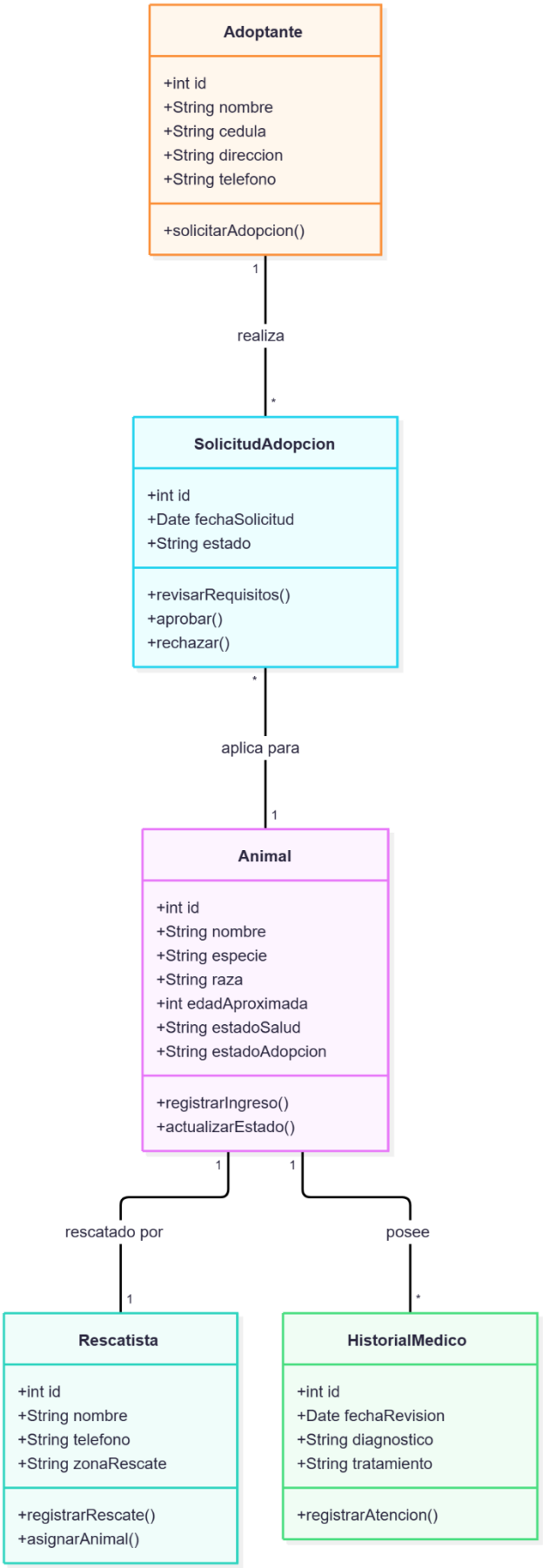
3. Modelado UML

A continuación, se presentan los diagramas de modelado que estructuran el comportamiento y la estructura estática del sistema.

3.1. Diagrama de Casos de Uso



3.2. Diagrama de Clases



4. Implementación de Patrones de Diseño

Para resolver problemas recurrentes de arquitectura y asegurar un código limpio, se han integrado tres patrones de diseño fundamentales:

4.1. Patrón Creacional: *Factory Method*

- **Aplicación:** Se utiliza para la creación dinámica de objetos de tipo `Animal` (por ejemplo, `Perro`, `Gato`, `Ave`) dependiendo de la especie ingresada por el rescatista en el momento del rescate. Esto desacopla la lógica de creación de clases concretas y facilita la incorporación de nuevas especies en el futuro sin modificar el código principal.

4.2. Patrón Estructural: *Singleton*

- **Aplicación:** Se implementa en la clase de gestión de conexión a la base de datos (`DatabaseConnection`). Garantiza que exista una única instancia de conexión activa durante toda la ejecución del sistema, optimizando recursos y evitando aperturas innecesarias de canales de comunicación con el servidor de datos.

4.3. Patrón de Comportamiento: *Observer*

- **Aplicación:** Se aplica en el proceso de solicitud de adopción. La clase `SolicitudAdopcion` actúa como el sujeto (*Subject*) y los usuarios adoptantes o el sistema de notificaciones actúan como observadores (*Observers*). Cuando el administrador cambia el estado de una solicitud (de "En revisión" a "Aprobada" o "Rechazada"), el sistema notifica automáticamente al adoptante vía correo o interfaz sin acoplar fuertemente ambas clases.

5. Conclusiones

El desarrollo de esta primera actividad permitió consolidar los conceptos teóricos y prácticos de la ingeniería de software moderna. El análisis previo y el modelado mediante UML facilitan una visión clara del alcance del sistema, mientras que la selección consciente de patrones de diseño asegura que la arquitectura sea escalable frente a futuros requerimientos. Asimismo, el dominio de Git y Gitflow fortalece las habilidades profesionales para trabajar de manera colaborativa y ordenada en proyectos tecnológicos.